当 n 个不同元素入栈时，出栈序列的个数为

$$\frac{1}{n+1}C_{2n}^{n}=\frac{1}{n+1}\frac{(2n)!}{n!\times n!}=\frac{6\times5\times4}{4\times3\times2\times1}=5$$

考题中给出的 n 值不会很大，可以根据栈的特点，若 X_i 已经出栈，则 X_i 前面的尚未出栈的元素一定逆置有序地出栈，因此可采用例举方法。如 a, b, c 依次入栈的出栈序列有 abc, acb, bac, bca, cba。另外，在一些考题中可能会问符合某个特定条件的出栈序列有多少种，比如此题中的问以 b 开头的出栈序列有几种，这种类型的题目一般都使用穷举法。

14．D

根据栈“先进后出”的特点，且在入栈操作的同时允许出栈操作，显然选项 D 中 c 最先出栈，则此时栈内必定为 a 和 b，但因为 a 先于 b 入栈，所以要晚出栈。对于某个出栈的元素，在它之前入栈却晚出栈的元素必定是按逆序出栈的，其余答案均是可能出现的情况。

此题也可采用将各序列逐个代入的方法来确定是否有对应的进出栈序列（类似下题）。

15．A

假设出栈序列为 $cd\cdots$，分析栈的操作序列：a 入栈，b 入栈，c 入栈，c 出栈，d 入栈，d 出栈，此后只能是 b 出栈和 a 出栈一种情况，因此出栈序列只有 $cdba$。

16．D

采用排除法，选项 A, B, C 得到的出栈序列分别为 1243, 3241, 1324。由 1234 得到 1342 的进出栈序列为：1 进，1 出，2 进，3 进，3 出，4 进，4 出，2 出，所以选择选项 D。

17．D

第 n 个元素第一个出栈，说明前 $n-1$ 个元素都已经按顺序入栈，由“先进后出”的特点可知，此时的输出序列一定是输入序列的逆序，所以答案选择选项 D。

18．D

当第 i 个元素第一个出栈时，则 i 之前的元素可以依次排在 i 之后出栈，但剩余的元素可以在此时入栈，并且排在 i 之前的元素出栈，所以第 j 个出栈的元素是不确定的。

19．C

对于 A，可能的顺序是 a 入，a 出，b 入，b 出，c 入，c 出，d 入，d 出。对于选项 B，可能的顺序是 a 入，b 入，c 入，c 出，b 出，d 入，d 出，a 出。对于选项 D，可能的顺序是 a 入，a 出，b 入，c 入，c 出，b 出，d 入，d 出。选项 C 没有对应的序列。

【另解】若出栈序列的第一个元素为 d，则出栈序列只能是 $dcba$。该思想通常也适用于出栈序列的局部分析：如 12345 入栈，问出栈序列 34152 是否正确？如何分析？若第一个出栈元素是 3，则此时 12 必停留在栈中，它们出栈的相对顺序只能是 21，所以 34152 错误。

20．C

入栈序列是 $P_1, P_2, \cdots, P_n$。因为 $P_3=1$，即 P_1, P_2, P_3 连续入栈后，第一个出栈元素是 P_3，说明 P_1, P_2 已经按序入栈，根据先进后出的特点可知，P_2 必定在 P_1 之前出栈，而第二个出栈元素是 2，而此时 P_1 不是栈顶元素，所以 P_1 的值不可能是 2。思考：哪些 P_i 可能是 2？

21．A

假设 P_1 是 1，入栈后立即出栈，P_2 是 2，入栈后立即出栈，P_3 是 3，入栈后立即出栈，得到的序列符合题意。假设 P_1 是 2，P_2 是 1，P_1、P_2 依次入栈后全部出栈，P_3 是 3，入栈后立即出栈，得到的序列符合题意。因此，P_1 既可能是 1，又可能是 2。

22．C

逐个判断每个选项可能的入栈出栈顺序。对于选项 A，可能的顺序是 1 入，1 出，2 入，2 出，

3 入，3 出，4 入，4 出。对于选项 B，可能的顺序是 1 入，2 入，3 入，3 出，2 出，4 入，4 出，1 出。对于选项 D，可能的顺序是 1 入，1 出，2 入，3 入，3 出，2 出，4 入，4 出。选项 C 没有对应的序列，因为当 4 在栈中时，意味着前面的所有元素（1, 2, 3）都已在栈中或曾经入过栈，此时若 4 第二个出栈，即栈中还有两个元素，且这两个元素是有序的（对应入栈顺序），只能为(1, 2), (1, 3), (2, 3)，若是序列(1, 2)，则 3 已在 p_1 位置出栈，不可能再在 p_4 位置出栈，若是(1, 3)和(2, 3)这种情况中的任意一种，则 3 一定是下一个出栈元素，即 p_3 一定是 3，所以 p_4 不可能是 3。

【另解】对于 C，p_2 为最后一个入栈元素 4，则只有 p_1 或 p_3 出栈的元素有可能为 3（请读者分两种情况自行思考），而 p_4 绝不可能为 3。读者在解答此类题时，一定要注意出栈序列中的“最后一个入栈元素”，这样可以节省答题的时间。

23．C

标识符只能以英文字母或下画线开头，而不能以数字开头。于上，由 n、1、_三个字符组合成的标识符有 n1_、n_1、_1n 和_n1 四种。第一种：n 入栈再出栈，1 入栈再出栈，_入栈再出栈。第二种：n 入栈再出栈，1 入栈，_入栈，_出栈，1 出栈。第三种：n 入栈，1 入栈，_入栈，_出栈，1 出栈，n 出栈。而根据栈的操作特性，_n1 这种情况不可能出现。

24．B

上溢是指存储器满，还往里写；下溢是指存储器空，还往外读。为了解决上溢，可给栈分配很大的存储空间，但这样又会造成存储空间的浪费。共享栈的提出就是为了在解决上溢的基础上节省存储空间，将两个栈放在同一段更大的存储空间内，这样，当一个栈的元素增加时，可使用另一个栈的空闲空间，从而降低发生上溢的可能性。

25．A

这种情况就是前面我们所描述的，详细内容请参见本节考点精析部分对共享栈的讲解。另外，读者可以思考当 top1 的初值为 0，top2 的初值为 n-1 时栈满的条件。

注　意

栈顶、队头与队尾的指针的定义是不唯一的，做题时务必仔细审题和思考。

26．C

时刻注意栈的特点是先进后出，下表是出入栈的详细过程。

序号	说明	栈内	栈外	序号	说明	栈内	栈外
1	*a* 入栈	*a*		8	*e* 入栈	*ae*	*bdc*
2	*b* 入栈	*ab*		9	*f* 入栈	*aef*	*bdc*
3	*b* 出栈	*a*	*b*	10	*f* 出栈	*ae*	*bdcf*
4	*c* 入栈	*ac*	*b*	11	*e* 出栈	*a*	*bdcfe*
5	*d* 入栈	*acd*	*b*	12	*a* 出栈		*bdcfea*
6	*d* 出栈	*ac*	*bd*	13	*g* 入栈	*g*	*bdcfea*
7	*c* 出栈	*a*	*bdc*	14	*g* 出栈		*bdcfeag*

栈内的最大深度为 3，所以栈 S 的容量至少是 3。

【另解】因为元素的出队顺序和入队顺序相同，所以元素的出栈顺序就是 *b*, *d*, *c*, *f*, *e*, *a*, *g*，因此元素的出入栈次序为 Push(S, *a*), Push(S, *b*), Pop(S, *b*), Push(S, *c*), Push(S, *d*), Pop(S, *d*), Pop(S, *c*), Push(S, *e*), Push(S, *f*), Pop(S, *f*), Pop(S, *e*), Pop(S, *a*), Push(S, *g*), Pop(S, *g*)。初始所需容量为 0，每做一次 Push 操作，容量加 1；每做一次 Pop 操作，容量减 1，记录的容量最大值为 3。

27. D

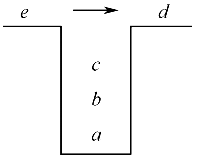
选项A由a入，b入，c入，d入，d出，c出，e入，e出，b出，f入，f出，a出得到；选项B由a入，b入，c入，c出，b出，d入，d出，a出，e入，e出，f入，f出得到；选项C由a入，b入，b出，c入，c出，a出，d入，e入，e出，f入，f出，d出得到；选项D由a入，a出，b入，c入，d入，e入，f入，f出，e出，d出，c出，b出得到，但题意要求不允许连续3次出栈操作，选项D不符。

【另解】先入栈的元素后出栈，入栈顺序为a, b, c, d, e, f，所以连续出栈时的子序列必然是按字母表逆序的，若出栈序列中出现了长度大于或等于3的连续逆序子序列，则为所选序列。

28. B

d第一个出栈，则c, b, a出栈的相对顺序是确定的，出栈顺序必为$d_c_b_a_$，e的顺序不定，在任意一个“_”上都有可能。

【另解】d首先出栈，则abc停留在栈中，此时栈的状态如下图所示。



此时可以有如下4种操作：①e入栈后出栈，则出栈序列为$decba$；②c出栈，e入栈后出栈，出栈序列为$dceba$；③cb出栈，e入栈后出栈，出栈序列为$dcbea$；④cba出栈，e入栈后出栈，出栈序列为$dcbae$。思路和上面其实一样。

29. C

显然，3之后的$4, 5, \cdots, n$都是P_3可取的数（一直入栈直到该数入栈后马上出栈）。接下来分析1和2是否可取：P_1可以是3之前入栈的数（可能是1或2），也可以是4，当$P_1=1$时，P_3可取2；当$P_1=2$时，P_3可取1。因此，p_3可能取除3外的所有数，个数为$n-1$。

30. D

按题意，出入栈操作的过程如下：

操作	栈内元素	出栈元素
Push	a	
Push	$a\ b$	
Pop	a	b
Push	$a\ c$	
Pop	a	c
Push	$a\ d$	
Push	$a\ d\ e$	
Pop	$a\ d$	e

因此，出栈序列为b, c, e。

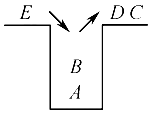
31. D

通过模拟出入栈操作，可以判断入栈序列in和出栈序列out是否合法。因此，已知in序列可以判断out序列是否为可能的出栈序列；已知out序列也可以判断in序列是否为可能的入栈序列，选项A和B错误。若每个元素入栈后立即出栈，则in序列和out序列相同，选项C错误。若所有元素都入栈后才依次出栈，则in序列和out序列互为倒序，选项D正确。

二、综合应用题

01.【解答】

CD 出栈后的状态如下图所示。



此时有如下 3 种操作：①*E* 入栈后出栈，出栈序列为 *CDEBA*；②*B* 出栈，*E* 入栈后出栈，出栈序列为 *CDBEA*；③*B* 出栈，*A* 出栈，*E* 入栈后出栈，出栈序列为 *CDBAE*。

所以，以 *CD* 开头的出栈序列有 *CDEBA*、*CDBEA*、*CDBAE* 三种。

02.【解答】

能得到出栈序列 *BCAED*。可由 *A* 入，*B* 入，*B* 出，*C* 入，*C* 出，*A* 出，*D* 入，*E* 入，*E* 出，*D* 出得到。不能得到出栈序列 *DBACE*。若出栈序列以 *D* 开头，说明在 *D* 之前的入栈元素是 *A*、*B* 和 *C*，三个元素中 *C* 是栈顶元素，*B* 和 *A* 不可能早于 *C* 出栈，所以不可能得到出栈序列 *DBACE*。

03.【解答】

1）选项 A、D 合法，而选项 B、C 不合法。在 B 中，先入栈 1 次，再连续出栈 2 次，错误。在选项 C 中，入栈和出栈次数不一致，会导致最终的栈不空。选项 A、D 均为合法序列，请自行模拟。注意，在操作过程中，入栈次数一定大于或等于出栈次数；结束时，栈一定为空。

2）设被判定的操作序列已存入一维数组 *A* 中。算法的基本设计思想：依次逐一扫描入栈出栈序列（由“I”和“O”组成的字符串），每扫描至任意一个位置均需检查出栈次数（“O”的个数）是否小于入栈次数（“I”的个数），若大于则为非法序列。扫描结束后，再判断入栈和出栈次数是否相等，若不相等则不合题意，为非法序列。

本题代码如下：

```
bool Judge(char A[ ]){
    int i=0;
    int j=k=0;                          //i 为下标，j 和 k 分别为字母 I 和 O 的个数
    while(A[i]!='\0'){                  //未到字符数组尾
        switch(A[i]){
            case 'I': j++; break;       //入栈次数增 1
            case 'O': k++;
                if(k>j){printf("序列非法\n"); exit(0); }
        }
        i++;                            //不论 A[i]是 I 还是 O，指针 i 均后移
    }
    if(j!=k){
        printf("序列非法\n");
        return  false;
    }
    else{
        printf("序列合法\n");
        return  true;
    }
}
```

【另解】入栈后，栈内元素个数加 1；出栈后，栈内元素个数减 1，因此可将判定一组出入栈序列是否合法转化为一组由+1、-1 组成的序列，它的任意前缀子序列的累加和不小于 0（每次出栈或入栈操作后判断）则合法；否则非法。

04.【解答】

算法思想：使用栈来判断链表中的数据是否中心对称。让链表的前一半元素依次入栈。在处

理链表的后一半元素时，当访问到链表的一个元素后，就从栈中弹出一个元素，对两个元素进行比较，若相等，则将链表中的下一个元素与栈中再弹出的元素进行比较，直至链表到尾。这时若栈是空栈，则得出链表中心对称的结论；否则，当链表中的一个元素与栈中弹出元素不等时，结论为链表非中心对称，结束算法的执行。

本题代码如下：

```
int dc(LinkList L,int n){
    int i;
    char s[n/2];                          //s 字符栈
    LNode *p=L->next;                     //工作指针 p，指向待处理的当前元素
    for(i=0;i<n/2;i++){                   //链表前一半元素入栈
        s[i]=p->data;
        p=p->next;
    }
    i--;                                  //恢复最后的 i 值
    if(n%2==1)                            //若 n 是奇数，后移过中心结点
        p=p->next;
    while(p!=NULL&&s[i]==p->data){        //检测是否中心对称
        i--;                              //i 充当栈顶指针
        p=p->next;
    }
    if(i==-1)                             //栈为空栈
        return  1;                        //链表中心对称
    else
        return  0;                        //链表中心不对称
}
```

算法先将“链表的前一半”元素（字符）入栈。当 n 为偶数时，前一半和后一半的个数相同；当 n 为奇数时，链表中心结点字符不必比较，移动链表指针到下一字符开始比较。比较过程中遇到不相等时，立即退出 while 循环，不再进行比较。

本题也可以先将单链表中的元素全部入栈，然后扫描单链表 L 并比较，直到比较到单链表 L 尾为止，但算法需要两次扫描单链表 L，效率不及上述算法高。

05.【解答】

两个栈共享向量空间，将两个栈的栈底设在向量两端，初始时，S1 栈顶指针为-1，S2 栈顶指针为 maxsize。两个栈顶指针相邻时为栈满。两个栈顶相向、迎面增长，栈顶指针指向栈顶元素。

本题代码如下：

```
#define  maxsize   100              //两个栈共享顺序存储空间所能达到的最多元素数，
                                    //初始化为 100
#define  elemtp int                 //假设元素类型为整型
typedef  struct{
    elemtp stack[maxsize];          //栈空间
    int   top[2];                   //top 为两个栈顶指针
}stk;
stk   s;                            //s 是如上定义的结构类型变量，为全局变量
```

本题的关键在于，两个栈入栈和出栈时的栈顶指针的计算。S1 栈是通常意义下的栈；而 S2 栈入栈操作时，其栈顶指针左移（减 1），出栈时，栈顶指针右移（加 1）。

此外，对于所有栈的操作，都要注意“入栈判满、出栈判空”的检查。

（1）入栈操作

代码如下：

```
int push(int i, elemtp x){
//入栈操作。i 为栈号，i=0 表示左边的 S1 栈，i=1 表示右边的 S2 栈，x 是入栈元素
```

```
    //入栈成功返回 1，否则返回 0
        if(i<0||i>1){
            printf("栈号输入不对");
            exit(0);
        }
        if(s.top[1]-s.top[0]==1){
            printf("栈已满\n");
            return  0;
        }
        switch(i){
            case 0: s.stack[++s.top[0]]=x; return  1; break;
            case 1: s.stack[--s.top[1]]=x; return  1;
        }
    }
```

（2）出栈操作

代码如下：

```
    elemtp pop(int i){
    //出栈算法。i 代表栈号，i=0 时为 S1 栈，i=1 时为 S2 栈
    //出栈成功返回出栈元素，否则返回-1
        if(i<0||i>1){
            printf("栈号输入错误\n");
            exit(0);
        }
        switch(i){
            case 0:
                if(s.top[0]==-1){
                    printf("栈空\n");
                    return  -1;
                }
                else
                    return  s.stack[s.top[0]--];
                break;
            case 1:
                if(s.top[1]==maxsize){
                    printf("栈空\n");
                    return  -1;
                }
                else
                    return  s.stack[s.top[1]++];
                break;
        }//switch
    }
```

3.2　队列

3.2.1　队列的基本概念

1．队列的定义

队列（Queue）简称队，也是一种操作受限的线性表，只允许在表的一端进行插入，而在表的另一端进行删除。向队列中插入元素称为入队或进队；删除元素称为出队或离队。这和我们日常生活中的排队是一致的，最早排队的也是最早离队的，其操作的特性是先进先出（First In First Out，FIFO），如图 3.5 所示。

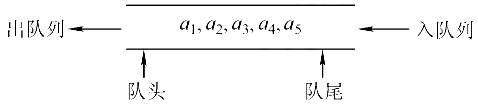


图 3.5 队列示意图

队头（Front）：允许删除的一端，也称队首。

队尾（Rear）：允许插入的一端。

空队列：不含任何元素的空表。

2．队列常见的基本操作

`InitQueue(&Q)`：初始化队列，构造一个空队列 `Q`。

`QueueEmpty(Q)`：判队列空，若队列 `Q` 为空返回 `true`，否则返回 `false`。

`EnQueue(&Q,x)`：入队，若队列 `Q` 未满，将 `x` 加入，使之成为新的队尾。

`DeQueue(&Q,&x)`：出队，若队列 `Q` 非空，删除队首元素，并用 `x` 返回。

`GetHead(Q,&x)`：读队首元素，若队列 `Q` 非空，则将队首元素赋值给 `x`。

需要注意的是，栈和队列是操作受限的线性表，因此不是任何对线性表的操作都可以作为栈和队列的操作。比如，不可以随便读取栈或队列中间的某个数据。

3.2.2 队列的顺序存储结构

1．队列的顺序存储

队列的顺序实现是指分配一块连续的存储单元存放队列中的元素，并附设两个指针：队首指针 `front` 指向队首元素，队尾指针 `rear` 指向队尾元素的下一个位置（不同教材对 `front` 和 `rear` 的定义可能不同，例如，可以让 `rear` 指向队尾元素、`front` 指向队首元素。对于不同的定义，出入队的操作是不同的，本节后面有一些相关的习题，读者可以结合习题思考）。

队列的顺序存储类型可描述为

```
#define MaxSize 50                  //定义队列中元素的最大个数
   typedef struct{
   ElemType data[MaxSize];          //用数组存放队列元素
   int front,rear;                  //队首指针和队尾指针
}SqQueue;
```

初始时：`Q.front=Q.rear=0`。

入队操作：队不满时，先送值到队尾元素，再将队尾指针加 1。

出队操作：队不空时，先取队首元素值，再将队首指针加 1。

图 3.6(a)所示为队列的初始状态，有 `Q.front==Q.rear==0` 成立，该条件可以作为队列判空的条件。

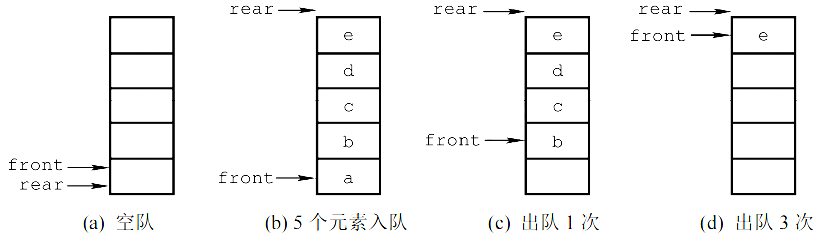


图 3.6 队列的操作

但能否用 `Q.rear==MaxSize` 作为队列满的条件呢？显然不能，图 3.6(d)中，队列中仅有一

个元素，但仍满足该条件。这时入队出现“上溢出”，但这种溢出并不是真正的溢出，在 data 数组中依然存在可以存放元素的空位置，所以是一种“假溢出”。

2．循环队列

上面指出了顺序队列“假溢出”的问题，这里引出循环队列的概念。将顺序队列臆造为一个环状的空间，即把存储队列元素的表从逻辑上视为一个环，称为循环队列。当队首指针 Q.front=MaxSize-1 后，再前进一个位置就自动到 0，这可以利用除法取模运算（%）来实现。

命题追踪 ▶ 特定条件下循环队列队头/队尾指针的初值（2011）

初始时：Q.front=Q.rear=0。

队首指针进 1：Q.front=(Q.front+1)%MaxSize。

队尾指针进 1：Q.rear=(Q.rear+1)%MaxSize。

队列长度：(Q.rear+MaxSize-Q.front)%MaxSize。

出入队时：指针都按顺时针方向进 1（如图 3.7 所示）。

命题追踪 ▶ 特定条件下循环队列队空/队满的判断条件（2014）

那么，循环队列队空和队满的判断条件是什么呢？显然，队空的条件是 Q.front==Q.rear。若入队元素的速度快于出队元素的速度，则队尾指针很快就会赶上队首指针，如图 3.7(d1)所示，此时可以看出队满时也有 Q.front==Q.rear。循环队列出入队示意图如图 3.7 所示。

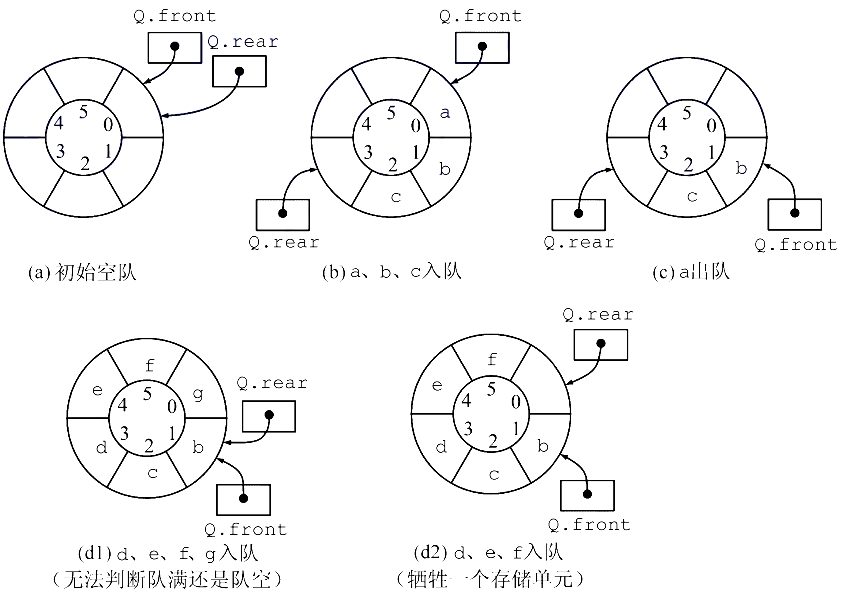


图 3.7　循环队列出入队示意图

为了区分是队空还是队满的情况，有三种处理方式：

1）牺牲一个单元来区分队空和队满，入队时少用一个队列单元，这是一种较为普遍的做法，约定以“队首指针在队尾指针的下一位置作为队满的标志”，如图 3.7(d2)所示。

队满条件：(Q.rear+1)%MaxSize==Q.front。

队空条件：Q.front==Q.rear。

队列中元素的个数：(Q.rear-Q.front+MaxSize)%MaxSize。

2）类型中增设 size 数据成员，表示元素个数。若删除成功，则 size 减 1，若插入成功，

则 size 加 1，队空时 Q.size==0；队满时 Q.size==MaxSize，两种情况都有 Q.front==Q.rear。

3）类型中增设 tag 数据成员，以区分是队满还是队空。删除成功置 tag=0，若导致 Q.front==Q.rear，则为队空；插入成功置 tag=1，若导致 Q.front==Q.rear，则为队满。

3．循环队列的操作

（1）初始化

```
void InitQueue(SqQueue &Q){
    Q.rear=Q.front=0;                    //初始化队首、队尾指针
}
```

（2）判队空

```
bool isEmpty(SqQueue Q){
    if(Q.rear==Q.front)                  //队空条件
        return true;
    else
        return  false;
}
```

（3）入队

```
bool EnQueue(SqQueue &Q,ElemType x){
    if((Q.rear+1)%MaxSize==Q.front)      //队满则报错
        return false;
    Q.data[Q.rear]=x;
    Q.rear=(Q.rear+1)%MaxSize;           //队尾指针加 1 取模
    return  true;
}
```

（4）出队

```
bool DeQueue(SqQueue &Q,ElemType &x){
    if(Q.rear==Q.front)                  //队空则报错
        return false;
    x=Q.data[Q.front];
    Q.front=(Q.front+1)%MaxSize;         //队首指针加 1 取模
    return  true;
}
```

3.2.3 队列的链式存储结构

1．队列的链式存储

命题追踪 ▶ 根据需求分析队列适合的存储结构（2019）

队列的链式表示称为链式队列，它实际上是一个同时有队首指针和队尾指针的单链表，如图 3.8 所示。队首指针指向队头结点，队尾指针指向队尾结点，即单链表的最后一个结点。

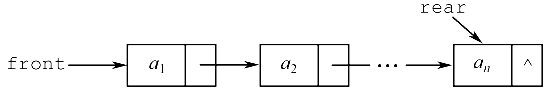


图 3.8 不带队头结点的链式队列

队列的链式存储类型可描述为

```
typedef struct LinkNode{                 //链式队列结点
    ElemType data;
    struct LinkNode *next;
}LinkNode;
```

```
typedef struct{                          //链式队列
    LinkNode *front,*rear;               //队列的队头和队尾指针
}LinkQueue;
```

不带头结点时，当 Q.front==NULL 且 Q.rear==NULL 时，链式队列为空。

命题追踪 ▶ 链式队列队空的判断（2019）

入队时，建立一个新结点，将新结点插入到链表的尾部，并让 Q.rear 指向这个新插入的结点（若原队列为空队，则令 Q.front 也指向该结点）。出队时，首先判断队是否为空，若不空，则取出队首元素，将其从链表中删除，并让 Q.front 指向下一个结点（若该结点为最后一个结点，则置 Q.front 和 Q.rear 都为 NULL）。

不难看出，不带头结点的链式队列在操作上往往比较麻烦，因此通常将链式队列设计成一个带头结点的单链表，这样插入和删除操作就统一了，如图 3.9 所示。

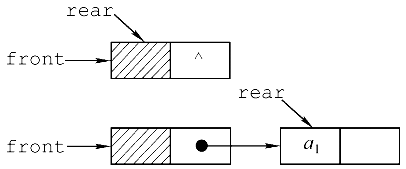


图 3.9　带队头结点的链式队列

用单链表表示的链式队列特别适合于数据元素变动比较大的情形，而且不存在队列满且产生溢出的问题。另外，假如程序中要使用多个队列，与多个栈的情形一样，最好使用链式队列，这样就不会出现存储分配不合理和“溢出”的问题。

2．链式队列的基本操作

命题追踪 ▶ 链式队列出队/入队操作的基本过程（2019）

（1）初始化

```
void InitQueue(LinkQueue &Q){   //初始化带头结点的链式队列
    Q.front=Q.rear=(LinkNode*)malloc(sizeof(LinkNode));//建立头结点
    Q.front->next=NULL;                 //初始为空
}
```

（2）判队空

```
bool IsEmpty(LinkQueue Q){
    if(Q.front==Q.rear)                 //判空条件
        return true;
    else
        return false;
}
```

（3）入队

```
void EnQueue(LinkQueue &Q,ElemType x){
    LinkNode *s=(LinkNode *)malloc(sizeof(LinkNode)); //创建新结点
    s->data=x;
    s->next=NULL;
    Q.rear->next=s;                     //插入链尾
    Q.rear=s;                           //修改尾指针
}
```

（4）出队

```
bool DeQueue(LinkQueue &Q,ElemType &x){
    if(Q.front==Q.rear)
        return  false;                  //空队
```